Application Security Verification Standard

(Level 2)

	Name	Date	Version
Created	Sai Pavan	26-06-2025	1.0
Reviewed	Gauri Shankar	30-06-2025	
Submission			

Tables of Contents

V1 Encoding and Sanitization

(Refer to pages 23–28 in the linked document)

V1.1 Encoding and Sanitization Architecture

ID	Description	Level	Reference
1.1.1	Verify that input is decoded or unescaped into a canonical form only once, it is only decoded when encoded data in that form is expected, and that this is done before processing the input further, for example, it is not performed after input validation or sanitization.	2	Link
1.1.2	Verify that the application performs output encoding and escaping either as a final step before being used by the interpreter for which it is intended or by the interpreter itself.	2	Link

V1.2 Injection Prevention

ID	Description	Level	Reference
1.2.6	Verify that the application protects against LDAP injection vulnerabilities, or that specific security controls to prevent LDAP injection have been implemented.	2	Link
1.2.7	Verify that the application is protected against XPath injection attacks by using query parameterization or precompiled queries.	2	Link
1.2.8	Verify that LaTeX processors are configured securely (such as not using the “shellescape” flag) and an allowlist of commands is used to prevent LaTeX injection attacks.	2	Link
1.2.9	Verify that the application escapes special characters in regular expressions (typically using a backslash) to prevent them from being misinterpreted as metacharacters.	2	Link

V1.3 Sanitization

ID	Description	Level	Reference
1.3.3	Verify that data being passed to a potentially dangerous context is sanitized beforehand to enforce safety measures, such as only allowing characters which are safe for this context and trimming input which is too long.	2	Link
1.3.4	Verify that user-supplied Scalable Vector Graphics (SVG) scriptable content is validated or sanitized to contain only tags and attributes (such as draw graphics) that are safe for the application, e.g., do not contain scripts and foreignObject.	2	Link
1.3.5	Verify that the application sanitizes or disables user-supplied scriptable or expression template language content, such as Markdown, CSS or XSL stylesheets, BBCode, or similar.	2	Link
1.3.6	Verify that the application protects against Server-Side Request Forgery (SSRF) attacks, by validating untrusted data against an allowlist of protocols, domains, paths, and ports, and sanitizing potentially dangerous characters before using the data to call another service.	2	Link
1.3.7	Verify that the application protects against template injection attacks by not allowing templates to be built based on untrusted input. Where there is no alternative, any untrusted input being included dynamically during template creation must be sanitized or strictly validated.	2	Link
1.3.8	Verify that the application appropriately sanitizes untrusted input before use in Java Naming and Directory Interface (JNDI) queries and that JNDI is configured securely to prevent JNDI injection attacks.	2	
1.3.9	Verify that the application sanitizes content before it is sent to memcache to prevent injection attacks.	2	
1.3.10	Verify that format strings which might resolve in an unexpected or malicious way when used are sanitized before being processed.	2	

1.3.11	Verify that the application sanitizes user input before passing to mail systems to protect against SMTP or IMAP injection.	2	

V1.4 Memory, String, and Unmanaged Code

ID	Description	Level	Reference
1.4.1	Verify that the application uses memory-safe string, safer memory copy, and pointer arithmetic to detect or prevent stack, buffer, or heap overflows.	2	Link
1.4.2	Verify that sign, range, and input validation techniques are used to prevent integer overflows.	2	
1.4.3	Verify that dynamically allocated memory and resources are released, and that references or pointers to freed memory are removed or set to null to prevent dangling pointers and use-after-free vulnerabilities.	2	

V1.5 Safe Deserialization

ID	Description	Level	Reference
1.5.2	Verify that deserialization of untrusted data enforces safe input handling, such as using an allowlist of object types or restricting client-defined object types, to prevent deserialization attacks. Deserialization mechanisms that are explicitly defined as insecure must not be used with untrusted input.	2	Link

V2 Validation and Business Logic

(Refer to pages 28-31 in the linked document)

V2.1 Validation and Business Logic Documentation

ID	Description	Level	Reference
2.1.2	Verify that the application's documentation defines how to validate the logical and contextual consistency of combined data items (e.g., suburb and ZIP code matching).	2	Link
2.1.3	Verify that expectations for business logic limits and validations are documented, including per-user and global expectations across the application.	2	

V2.2 Input Validation

ID	Description	Level	Reference
2.2.3	Verify that the application ensures that combinations of related data items are reasonable according to the predefined rules.	2	Link

V2.3 Business Logic Security

ID	Description	Level	Reference
2.3.2	Verify that business logic limits are implemented per the application's documentation to avoid business logic flaws being exploited.	2	Link
2.3.3	Verify that transactions are used at the business logic level such that a business logic operation succeeds in its entirety or is rolled back to the previous correct state.	2	
2.3.4	Verify that business logic level locking mechanisms are used to ensure that limited quantity resources (e.g., theater seats, delivery slots) cannot be double-booked.	2	

V2.4 Anti-automation

ID	Description	Level	Reference
2.4.1	Verify that anti-automation controls are in place to protect against excessive calls to application functions that could lead to data exfiltration, garbage data creation, quota exhaustion, rate-limit breaches, denial-of-service, or overuse of costly resources.	2	Link

V3 Web Frontend Security

(Refer to pages 32-37 in the linked document)

V3.3 – Cookie Setup

ID	Description	Level	Reference
3.3.2	Verify cookies set SameSite based on purpose to prevent CSRF and UI redress attacks.	2	Link
3.3.3	Verify cookies use __Host- prefix unless they are designed to be shared across hosts.	2	
3.3.4	Verify cookies not meant for scripts have HttpOnly set, and values transferred only via Set-Cookie.	2	

V3.4 – Browser Security Mechanism Headers

ID	Description	Level	Reference
3.4.3	Ensure Content-Security-Policy has global directives like object-src 'none', base-uri 'none', and uses allowlists, nonces, or hashes.	2	Link
3.4.4	Include X-Content-Type-Options: nosniff to prevent MIME type guessing.	2	Link
3.4.5	Ensure the application sets a **Referrer-Policy** to prevent leakage of sensitive data (e.g., path, query parameters, or hostname) via the **'Referer'** HTTP header. This can be done using the **Referrer-Policy HTTP response header** or	2	

	HTML element attributes. Sensitive data should not be exposed to third-party services unnecessarily.		
3.4.6	Verify that the web application uses the **frame-ancestors** directive in the **Content-Security-Policy** header for every HTTP response to prevent unauthorized embedding and allow it only when necessary. Note that **X-Frame-Options**, though supported, is obsolete and should not be relied upon	2	

V3.5 – Browser Origin Separation

ID	Description	Level	Reference
3.5.4	Verify that separate applications are hosted on different hostnames to leverage the restrictions provided by same-origin policy, including how documents or scripts loaded by one origin can interact with resources from another origin and hostname-based restrictions on cookies.	2	Link
3.5.5	Verify that messages received by the postMessage interface are discarded if the origin of the message is not trusted, or if the syntax of the message is invalid.	2	

V3.7 – Other Browser Security Considerations

ID	Description	Level	Reference
3.7.1	Verify that the application only uses client-side technologies which are still supported and considered secure. Examples of technologies which do not meet this requirement include NSAPI plugins, Flash, Shockwave, ActiveX, Silverlight, NACL, or client-side Java applets	2	Link
3.7.2	Verify that the application will only automatically redirect the user to a different hostname or domain (which is not controlled by the application) where the destination appears on an allowlist.	2	

V4: API and Web Service Security

(Refer to pages 38 - 41 in the linked document)

V4.1 – Generic Web Service Security

ID	Description	Level
4.1.2	Verify that only user-facing endpoints (intended for manual web-browser access) automatically redirect from HTTP to HTTPS, while other services or endpoints do not implement transparent redirects. This is to avoid situations where clients mistakenly send unencrypted HTTP requests, and the automatic redirect to HTTPS leads to undetected leakage of sensitive data.	2
4.1.3	Verify that any HTTP header field used by the application and set by an intermediary layer (such as a load balancer, web proxy, or backend-for-frontend service) cannot be overridden by the end-user. Example headers include X-Real-IP, X-Forwarded-*, or X-User-ID.	2

V4.2 HTTP Message Structure Validation

ID	Description	Level
4.2.1	Verify that all application components (including load balancers, firewalls, and application servers) determine boundaries of incoming HTTP messages using the appropriate mechanism for the HTTP version to prevent HTTP request smuggling. In HTTP/1.x, if a Transfer-Encoding header field is present, the Content-Length header must be ignored per RFC 2616. When using HTTP/2 or HTTP/3, if a Content-Length header field is present, the receiver must ensure that it is consistent with the length of the DATA frames.	2

V4.3 GraphQL

ID	Description	Level	Reference
4.3.1	Verify that a query allowlist, depth limiting, amount limiting, or query cost analysis is used to prevent GraphQL or data layer expression Denial of Service (DoS) due to expensive or nested queries.	2	Link
4.3.2	Verify that GraphQL introspection queries are disabled in the production environment unless the GraphQL API is meant to be used by other parties.	2	

V4.4 WebSocket

ID	Description	Level	Reference
4.4.2	Verify that, during the initial HTTP WebSocket handshake, the Origin header field is checked against a list of origins allowed for the application.	2	Link
4.4.3	Verify that, if the application's standard session management cannot be used, dedicated tokens are being used for this, which comply with the relevant Session Management security requirements.	2	
4.4.4	Verify that dedicated WebSocket session management tokens are initially obtained or validated through the previously authenticated HTTPS session when transitioning an existing HTTPS session to a WebSocket channel.	2	

V5 File Handling

(Refer to pages 41-43 in the linked document)

V5.1 File Handling Documentation

ID	Description	Level	Reference
5.1.1	Verify that the documentation defines the permitted file types, expected file extensions, and maximum size (including unpacked size) for each upload feature. Additionally, ensure that the documentation specifies how files are made safe for end-users to download and process, such as how the application behaves when a malicious file is detected.	2	Link

V5.2 File Upload and Content

ID	Description	Level	Reference
5.2.3	Verify that the application checks compressed files (e.g., zip, gz, docx, odt) against maximum allowed uncompressed size and against the maximum number of files before uncompressing the file.	2	Link

V5.4 File Download

ID	Description	Level	Reference
5.4.1	Verify that the application validates or ignores user-submitted filenames, including in a JSON, JSONP, or URL parameter, and specifies a filename in the Content-Disposition header field in the response.	2	Link
5.4.2	Verify that file names served (e.g., in HTTP response header fields or email attachments) are encoded or sanitized (e.g., following RFC 6266) to preserve document structure and prevent injection attacks.	2	
5.4.3	Verify that files obtained from untrusted sources are scanned by antivirus scanners to prevent serving of known malicious content.	2	

V6 Authentication

(Refer to pages 44 – 52 in the linked document)

V6.1 Authentication Documentation

ID	Description	Level	Reference
6.1.2	Verify that a list of context-specific words is documented to prevent their use in passwords. The list could include permutations of organization names, product names, system identifiers, project codenames, department or role names, and similar.	2	Link
6.1.3	Verify that, if the application includes multiple authentication pathways, these are all documented together with the security controls and authentication strength which must be consistently enforced across them.	2	

V6.2 Password Security

ID	Description	Level	Reference
6.2.9	Verify that passwords of at least 64 characters are permitted.	2	Link
6.2.10	Verify that a user's password stays valid until it is discovered to be compromised or the user rotates it.	2	

	The application must not require periodic credential rotation.		
6.2.11	Verify that the documented list of context-specific words is used to prevent easy-to-guess passwords from being created.	2	
6.2.12	Verify that passwords submitted during account registration or password changes are checked against a set of breached passwords.	2	

V6.3 General Authentication Security

ID	Description	Level	Reference
6.3.3	Verify that either a multi-factor authentication mechanism or a combination of single-factor authentication mechanisms must be used in order to access the application. For L3, one of the factors must be a hardware-based authentication mechanism which provides compromise and impersonation resistance against phishing attacks while verifying the intent to authenticate by requiring a user-initiated action (such as a button press on a FIDO hardware key or a mobile phone). Relaxing any of the considerations in this requirement requires a fully documented rationale and a comprehensive set of mitigating controls.	2	Link
6.3.4	Verify that, if the application includes multiple authentication pathways, there are no undocumented pathways and that security controls and authentication strength are enforced consistently.	2	

V6.4 Authentication Factor Lifecycle and Recovery

ID	Description	Level	Reference
6.4.3	Verify that a secure process for resetting a forgotten password is implemented, that does not bypass any enabled multi-factor authentication mechanisms.	2	Link
6.4.4	Verify that if a multi-factor authentication factor is lost, evidence of identity proofing is performed at the same level as during enrollment.	2	

V6.5 General Multi-factor Authentication Requirements

ID	Description	Level	Reference

6.5.1	Verify that lookup secrets, out-of-band authentication requests or codes, and time-based one-time passwords (TOTPs) are only successfully usable once.	2	Link
6.5.2	Verify that, when being stored in the application's backend, lookup secrets with less than 112 bits of entropy (19 random alphanumeric characters or 34 random digits) are hashed with an approved password storage hashing algorithm that incorporates a 32-bit random salt. A standard hash function can be used if the secret has 112 bits of entropy or more.	2	
6.5.3	Verify that lookup secrets, out-of-band authentication code, and time-based one-time password seeds, are generated using a Cryptographically Secure Pseudorandom Number Generator (CSPRNG) to avoid predictable values.	2	
6.5.4	Verify that lookup secrets and out-of-band authentication codes have a minimum of 20 bits of entropy (typically 4 random alphanumeric characters or 6 random digits is sufficient).	2	
6.5.5	Verify that out-of-band authentication requests, codes, or tokens, as well as time-based one-time passwords (TOTPs) have a defined lifetime. Out-of-band requests must have a maximum lifetime of 10 minutes, and for TOTP a maximum lifetime of 30 seconds.	2	

V6.6 Out-of-Band Authentication Mechanisms

ID	Description	Level	Reference
6.6.1	Verify that authentication mechanisms using the Public Switched Telephone Network (PSTN) to deliver One-time Passwords (OTPs) via phone or SMS are offered only when the phone number has previously been validated, alternate stronger methods (such as Time based One-time Passwords) are also offered, and the service provides information on their security risks to users. For L3 applications, phone and SMS must not be available as options.	2	Link
6.6.2	Verify that out-of-band authentication requests, codes, or tokens are bound to the original authentication request for which they were generated and are not usable for a previous or subsequent one.	2	
6.6.3	Verify that a code based out-of-band authentication mechanism is protected against brute force attacks by using rate limiting. Consider also using a code with at least 64 bits of entropy.	2	

V6.8 Authentication with an Identity Provider

ID	Description	Level	Reference
6.8.1	Verify that, if the application supports multiple identity providers (IdPs), the user's identity cannot be spoofed via another supported identity provider (e.g., by using the same user identifier). The standard mitigation would be for the application to register and identify the user using a combination of the IdP ID (serving as a namespace) and the user's ID in the IdP.	2	Link
6.8.2	Verify that the presence and integrity of digital signatures on authentication assertions (for example on JWTs or SAML assertions) are always validated, rejecting any assertions that are unsigned or have invalid signatures.	2	
6.8.3	Verify that SAML assertions are uniquely processed and used only once within the validity period to prevent replay attacks.	2	
6.8.4	Verify that, if an application uses a separate Identity Provider (IdP) and expects specific authentication strength, methods, or recentness for specific functions, the application verifies this using the information returned by the IdP. For example, if OIDC is used, this might be achieved by validating ID Token claims such as 'acr', 'amr', and 'auth_time' (if present). If the IdP does not provide this information, the application must have a documented fallback approach that assumes that the minimum strength authentication mechanism was used (for example, single-factor authentication using username and password).	2	

V7 Session Management

(Refer to pages 52 - 56 in the linked document)

V7.1 Session Management Documentation

ID	Description	Level	Reference
7.1.1	Verify that the user's session inactivity timeout and absolute maximum session lifetime are documented, are	2	Link 1

	appropriate in combination with other controls, and that the documentation includes justification for any deviations from NIST SP 800-63B re-authentication requirements.		Link 2
7.1.2	Verify that the documentation defines how many concurrent (parallel) sessions are allowed for one account as well as the intended behaviors and actions to be taken when the maximum number of active sessions is reached.	2	
7.1.3	Verify that all systems that create and manage user sessions as part of a federated identity management ecosystem (such as SSO systems) are documented along with controls to coordinate session lifetimes, termination, and any other conditions that require re-authentication.	2	

V7.3 Session Timeout

ID	Description	Level	Reference
7.3.1	Verify that there is an inactivity timeout such that re-authentication is enforced according to risk analysis and documented security decisions.	2	Link
7.3.2	Verify that there is an absolute maximum session lifetime such that re-authentication is enforced according to risk analysis and documented security decisions.	2	Link

V7.4 Session Termination

ID	Description	Level	Reference
7.4.3	Verify that the application gives the option to terminate all other active sessions after a successful change or removal of any authentication factor (including password change via reset or recovery and, if present, an MFA settings update).	2	Link Link
7.4.4	Verify that all pages that require authentication have easy and visible access to logout functionality.	2	
7.4.5	Verify that application administrators can terminate active sessions for an individual user or for all users.	2	

V7.5 Defenses Against Session Abuse

ID	Description	Level	Reference

7.5.1	Verify that the application requires full re-authentication before allowing modifications to sensitive account attributes which may affect authentication such as email address, phone number, MFA configuration, or other information used in account recovery.	2	Link Link Link
7.5.2	Verify that users are able to view and (having authenticated again with at least one factor) terminate any or all currently active sessions.	2	

V7.6 Federated Re-authentication

ID	Description	Level	Reference
7.6.1	Verify that session lifetime and termination between Relying Parties (RPs) and Identity Providers (IdPs) behave as documented, requiring re-authentication as necessary such as when the maximum time between IdP authentication events is reached.	2	Link
7.6.2	Verify that creation of a session requires either the user's consent or an explicit action, preventing the creation of new application sessions without user interaction.	2	

V8 Authorization

(Refer to pages 56 - 59 in the linked document)

V8.1 Authorization Documentation

ID	Description	Level	Reference
8.1.2	Verify that authorization documentation defines rules for field-level access restrictions (both read and write) based on consumer permissions and resource attributes. Note that these rules might depend on other attribute values of the relevant data object, such as state or status.	2	Link Link

V8.2 General Authorization Design

ID	Description	Level	Reference
8.2.3	Verify that the application ensures that field-level access is restricted to consumers with explicit	2	Link

	permissions to specific fields to mitigate broken object property level authorization (BOPLA).		

V8.4 Other Authorization Considerations

ID	Description	Level	Reference
8.4.1	Verify that multi-tenant applications use cross-tenant controls to ensure consumer operations will never affect tenants with which they do not have permissions to interact.	2	Link

V9 Self-contained Tokens

(Refer to pages 59 – 61 in the linked document)

V9.2 Token Content

ID	Description	Level	Reference
9.2.2	Verify that the service receiving a token validates the token to be the correct type and is meant for the intended purpose before accepting the token's contents. For example, only access tokens can be accepted for authorization decisions and only ID Tokens can be used for proving user authentication.	2	Link
9.2.3	Verify that the service only accepts tokens which are intended for use with that service (audience). For JWTs, this can be achieved by validating the 'aud' claim against an allowlist defined in the service.	2	
9.2.4	Verify that, if a token issuer uses the same private key for issuing tokens to different audiences, the issued tokens contain an audience restriction that uniquely identifies the intended audiences. This will prevent a token from being reused with an unintended audience. If the audience identifier is dynamically provisioned, the token issuer must validate these audiences in order to make sure that they do not result in audience impersonation.	2	

V10 OAuth and OIDC

(Refer to pages 61 - 69 in the linked document)

V10.1 Generic OAuth and OIDC Security

ID	Description	Level	Reference
10.1.1	Verify that tokens are only sent to components that strictly need them. For example, when using a backend-for-frontend pattern for browser-based JavaScript applications, access and refresh tokens shall only be accessible for the backend.	2	Link
10.1.2	Verify that the client only accepts values from the authorization server (such as the authorization code or ID Token) if these values result from an authorization flow that was initiated by the same user agent session and transaction. This requires that client-generated secrets, such as the proof key for code exchange (PKCE) 'code_verifier', 'state' or OIDC 'nonce', are not guessable, are specific to the transaction, and are securely bound to both the client and the user agent session in which the transaction was started.	2	

V10.2 OAuth Client:

ID	Description	Level	Reference
10.2.1	Verify that, if the code flow is used, the OAuth client has protection against browser-based request forgery attacks, commonly known as cross-site request forgery (CSRF), which trigger token requests, either by using proof key for code exchange (PKCE) functionality or checking the 'state' parameter that was sent in the authorization request.	2	Link
10.2.2	Verify that, if the OAuth client can interact with more than one authorization server, it has a defense against mix-up attacks. For example, it could require that the authorization server return the 'iss' parameter value and validate it in the authorization response and the token response.	2	

V10.3 OAuth Resource Server

ID	Description	Level	Reference

10.3.1	Verify that the resource server only accepts access tokens that are intended for use with that service (audience). The audience may be included in a structured access token (such as the 'aud' claim in JWT), or it can be checked using the token introspection endpoint.	2	Link
10.3.2	Verify that the resource server enforces authorization decisions based on claims from the access token that define delegated authorization. If claims such as 'sub', 'scope', and 'authorization_details' are present, they must be part of the decision.	2	
10.3.3	Verify that if an access control decision requires identifying a unique user from an access token (JWT or related token introspection response), the resource server identifies the user from claims that cannot be reassigned to other users. Typically, it means using a combination of 'iss' and 'sub' claims.	2	
10.3.4	Verify that, if the resource server requires specific authentication strength, methods, or recentness, it verifies that the presented access token satisfies these constraints. For example, if present, using the OIDC 'acr', 'amr', and 'auth_time' claims respectively.	2	

V10.4 OAuth Authorization Server

ID	Description	Level	Reference
10.4.6	Verify that, if the code grant is used, the authorization server mitigates authorization code interception attacks by requiring proof key for code exchange (PKCE). For authorization requests, the authorization server must require a valid 'code_challenge' value and must not accept a 'code_challenge_method' value of 'plain'. For a token request, it must require validation of the 'code_verifier' parameter.	2	Link
10.4.7	Verify that if the authorization server supports unauthenticated dynamic client registration, it mitigates the risk of malicious client applications. It must validate client metadata such as any registered URIs, ensure the user's consent, and warn the user before processing an authorization request with an untrusted client application.	2	
10.4.8	Verify that refresh tokens have an absolute expiration, including if sliding refresh token expiration is applied.	2	
10.4.9	Verify that refresh tokens and reference access tokens can be revoked by an authorized user using the authorization server user interface, to mitigate the risk of malicious clients or stolen tokens.	2	

10.4.10	Verify that confidential clients are authenticated for client-to-authorization server backchannel requests such as token requests, pushed authorization requests (PAR), and token revocation requests.	2	
10.4.11	Verify that the authorization server configuration only assigns the required scopes to the OAuth client.	2	

V10.5 OIDC Client

ID	Description	Level	Reference
10.5.1	Verify that the client (as the relying party) mitigates ID Token replay attacks. For example, by ensuring that the 'nonce' claim in the ID Token matches the 'nonce' value sent in the authentication request to the OpenID Provider (in OAuth2 referred to as the authorization request sent to the authorization server).	2	Link
10.5.2	Verify that the client uniquely identifies the user from ID Token claims, usually the 'sub' claim, which cannot be reassigned to other users (for the scope of an identity provider).	2	
10.5.3	Verify that the client rejects attempts by a malicious authorization server to impersonate another authorization server through authorization server metadata. The client must reject authorization server metadata if the issuer URL in the authorization server metadata does not exactly match the pre-configured issuer URL expected by the client.	2	
10.5.4	Verify that the client validates that the ID Token is intended to be used for that client (audience) by checking that the 'aud' claim from the token is equal to the 'client_id' value for the client.	2	
10.5.5	Verify that, when using OIDC back-channel logout, the relying party mitigates denial of service through forced logout and cross-JWT confusion in the logout flow. The client must verify that the logout token is correctly typed with a value of 'logout+jwt', contains the 'event' claim with the correct member name, and does not contain a 'nonce' claim. Note that it is also recommended to have a short expiration (e.g., 2 minutes).	2	

V10.6 OpenID Provider

ID	Description	Level	Reference
10.6.1	Verify that the OpenID Provider only allows values 'code', 'ciba', 'id_token', or 'id_token code' for response mode. Note that 'code' is preferred over 'id_token code' (the OIDC Hybrid flow), and 'token' (any Implicit flow) must not be used.	2	Link
10.6.2	Verify that the OpenID Provider mitigates denial of service through forced logout. By obtaining explicit confirmation from the end-user or, if present, validating parameters in the logout request (initiated by the relying party), such as the 'id_token_hint'.	2	

V10.7 Consent Management

ID	Description	Level	Reference
10.7.1	Verify that the authorization server ensures that the user consents to each authorization request. If the identity of the client cannot be assured, the authorization server must always explicitly prompt the user for consent.	2	Link
10.7.2	Verify that when the authorization server prompts for user consent, it presents sufficient and clear information about what is being consented to. When applicable, this should include the nature of the requested authorizations (typically based on scope, resource server, Rich Authorization Requests (RAR) authorization details), the identity of the authorized application, and the lifetime of these authorizations.	2	
10.7.3	Verify that the user can review, modify, and revoke consents which the user has granted through the authorization server.	2	

V11 Cryptography

(Refer to pages 70 - 75 in the linked document)

V11.1 Cryptographic Inventory and Documentation

ID	Description	Level	Reference
11.1.1	Verify that there is a documented policy for management of cryptographic keys and a cryptographic key lifecycle that follows a key management standard such as NIST SP 800-57. This should include ensuring that keys are not overshared (for example, with more than two entities for shared secrets and more than one entity for private keys).	2	Link Link
11.1.2	Verify that a cryptographic inventory is performed, maintained, regularly updated, and includes all cryptographic keys, algorithms, and certificates used by the application. It must also document where keys can and cannot be used in the system, and the types of data that can and cannot be protected using the keys.	2	Link Link

V11.2 Secure Cryptography Implementation

ID	Description	Level	Reference
11.2.1	Verify that industry-validated implementations (including libraries and hardware-accelerated implementations) are used for cryptographic operations.	2	Link Link
11.2.2	Verify that the application is designed with crypto agility such that random number, authenticated encryption, MAC, or hashing algorithms, key lengths, rounds, ciphers and modes can be reconfigured, upgraded, or swapped at any time, to protect against cryptographic breaks. Similarly, it must also be possible to replace keys and passwords and re-encrypt data. This will allow for seamless upgrades to post-quantum cryptography (PQC), once high-assurance implementations of approved PQC schemes or standards are widely available.	2	Link Link
11.2.3	Verify that all cryptographic primitives utilize a minimum of 128-bits of security based on the algorithm, key size, and configuration. For example, a 256-bit ECC key provides roughly 128 bits of security where RSA requires a 3072-bit key to achieve 128 bits of security.	2	Link Link

V11.3 Encryption Algorithms

ID	Description	Level	Reference
11.3.3	Verify that encrypted data is protected against unauthorized modification, preferably by using an approved authenticated encryption method or by combining an approved encryption method with an approved MAC algorithm.	2	Link

V11.4 Hashing and Hash-based Functions

ID	Description	Level	Reference
11.4.2	Verify that passwords are stored using an approved, computationally intensive, key derivation function (also known as a "password hashing function"), with parameter settings configured based on current guidance. The settings should balance security and performance to make brute-force attacks sufficiently challenging for the required level of security.	2	Link
11.4.3	Verify that hash functions used in digital signatures, as part of data authentication or data integrity are collision resistant and have appropriate bit-lengths. If collision resistance is required, the output length must be at least 256 bits. If only resistance to second pre-image attacks is required, the output length must be at least 128 bits.	2	Link
11.4.4	Verify that the application uses approved key derivation functions with key stretching parameters when deriving secret keys from passwords. The parameters in use must balance security and performance to prevent brute-force attacks from compromising the resulting cryptographic key.	2	

V11.5 Random Values

ID	Description	Level	Reference
11.5.1	Verify that all random numbers and strings which are intended to be non-guessable must be generated using a cryptographically secure pseudo-random number generator (CSPRNG) and have at least 128 bits of entropy. Note that UUIDs do not respect this condition.	2	Link

11.5.2	Verify that the random number generation mechanism in use is designed to work securely, even under heavy demand.	3	

V11.6 Public Key Cryptography

ID	Description	Level	Reference
11.6.1	Verify that only approved cryptographic algorithms and modes of operation are used for key generation and seeding, and digital signature generation and verification. Key generation algorithms must not generate insecure keys vulnerable to known attacks, for example, RSA keys which are vulnerable to Fermat factorization.	2	Link

V12 Secure Communication

(Refer to pages 75 – 77 in the linked document)

V12.1 General TLS Security Guidance

ID	Description	Level	Reference
12.1.2	Verify that only recommended cipher suites are enabled, with the strongest cipher suites set as preferred. L3 applications must only support cipher suites which provide forward secrecy.	2	Link
12.1.3	Verify that the application validates that mTLS client certificates are trusted before using the certificate identity for authentication or authorization.	2	

V12.3 General Service-to-Service Communication Security

ID	Description	Level	Reference
12.3.1	Verify that an encrypted protocol such as TLS is used for all inbound and outbound connections to and from the application, including monitoring systems, management tools, remote access and SSH, middleware, databases, mainframes, partner systems, or external APIs. The server must not fall back to insecure or unencrypted protocols.	2	Link

12.3.2	Verify that TLS clients validate certificates received before communicating with a TLS server.	2	
12.3.3	Verify that TLS or another appropriate transport encryption mechanism is used for all connectivity between internal, HTTP-based services within the application, and does not fall back to insecure or unencrypted communications.	2	
12.3.4	Verify that TLS connections between internal services use trusted certificates. Where internally generated or self-signed certificates are used, the consuming service must be configured to only trust specific internal CAs and specific self-signed certificates.	2	

V13 Configuration

(Refer to pages 77–81 in the linked document)

V13.1 Configuration Documentation

ID	Description	Level	Reference
13.1.1	Verify that all communication needs for the application are documented. This must include external services which the application relies upon and cases where an end user might be able to provide an external location to which the application will then connect.	2	Link
13.1.2	Verify that for each service the application uses, the documentation defines the maximum number of concurrent connections (e.g., connection pool limits) and how the application behaves when that limit is reached, including any fallback or recovery mechanisms, to prevent denial of service conditions.	2	
13.1.3	Verify that the application documentation defines resource-management strategies for every external system or service it uses (e.g., databases, file handles, threads, HTTP connections). This should include resource-release procedures, timeout settings, failure handling, and where retry logic is implemented, specifying retry limits, delays, and back-off algorithms. For synchronous HTTP request–response operations it should mandate short timeouts and either disable retries or strictly limit retries to prevent cascading delays and resource exhaustion.	2	

V13.2 Backend Communication Configuration

ID	Description	Level	Reference
13.2.1	Verify that communications between backend application components that don't support the application's standard user session mechanism, including APIs, middleware, and data layers, are authenticated. Authentication must use individual service accounts, short-term tokens, or certificate-based authentication and not unchanging credentials such as passwords, API keys, or shared accounts with privileged access.	2	Link
13.2.2	Verify that communications between backend application components, including local or operating system services, APIs, middleware, and data layers, are performed with accounts assigned the least necessary privileges.	2	
13.2.3	Verify that if a credential has to be used for service authentication, the credential being used by the consumer is not a default credential (e.g., root/root or admin/admin).	2	
13.2.4	Verify that an allowlist is used to define the external resources or systems with which the application is permitted to communicate (e.g., for outbound requests, data loads, or file access). This allowlist can be implemented at the application layer, web server, firewall, or a combination of different layers.	2	
13.2.5	Verify that the web or application server is configured with an allowlist of resources or systems to which the server can send requests or load data or files from.	2	

V13.3 Secret Management

ID	Description	Level	Reference
13.3.1	Verify that a secrets management solution, such as a key vault, is used to securely create, store, control access to, and destroy backend secrets. These could include passwords, key material, integrations with databases and third-party systems, keys and seeds for time-based tokens, other internal secrets, and API keys. Secrets must not be included in application source code or included in build artifacts. For an L3 application, this must involve a hardware-backed solution such as an HSM.	2	Link
13.3.2	Verify that access to secret assets adheres to the principle of least privilege.	2	

V13.4 Unintended Information Leakage

ID	Description	Level	Reference
13.4.2	Verify that debug modes are disabled for all components in production environments to prevent exposure of debugging features and information leakage.	2	Link
13.4.3	Verify that web servers do not expose directory listings to clients unless explicitly intended.	2	
13.4.4	Verify that using the HTTP TRACE method is not supported in production environments, to avoid potential information leakage.	2	
13.4.5	Verify that documentation (such as for internal APIs) and monitoring endpoints are not exposed unless explicitly intended.	2	

V14 Data Protection

(Refer to pages 81- 84 in the linked document)

V14.1 Data Protection Documentation

ID	Description	Level	Reference
14.1.1	Verify that all sensitive data created and processed by the application has been identified and classified into protection levels. This includes data that is only encoded and therefore easily decoded, such as Base64 strings or the plaintext payload inside a JWT. Protection levels need to take into account any data protection and privacy regulations and standards which the application is required to comply with.	2	Link
14.1.2	Verify that all sensitive data protection levels have a documented set of protection requirements. This must include (but not be limited to) requirements related to general encryption, integrity verification, retention, how the data is to be logged, access controls around sensitive data in logs, database-level encryption, privacy and privacy-enhancing technologies to be used, and other confidentiality requirements.	2	

V14.2 General Data Protection

ID	Description	Level	Reference
14.2.2	Verify that the application prevents sensitive data from being cached in server components, such as load balancers and application caches, or ensures that the data is securely purged after use.	2	Link
14.2.3	Verify that defined sensitive data is not sent to untrusted parties (e.g., user trackers) to prevent unwanted collection of data outside of the application's control.	2	Link
14.2.4	Verify that controls around sensitive data related to encryption, integrity verification, retention, how the data is to be logged, access controls around sensitive data in logs, privacy and privacy-enhancing technologies, are implemented as defined in the documentation for the specific data's protection level.	2	
14.2.5	Verify that caching mechanisms are configured to only cache responses which have the expected content type for that resource and do not contain sensitive, dynamic content. The web server should return a 404 or 302 response when a non-existent file is accessed rather than returning a different, valid file. This should prevent Web Cache Deception attacks.	2	

V14.3 Client-Side Data Protection

ID	Description	Level	Reference
14.3.2	Verify that the application sets sufficient anti-caching HTTP response header fields (i.e., Cache-Control: no-store) so that sensitive data is not cached in browsers.	2	Link
14.3.3	Verify that data stored in browser storage (such as localStorage, sessionStorage, IndexedDB, or cookies) does not contain sensitive data, apart from session tokens.	2	Link

V15 Secure Coding and Architecture

(Refer to pages 84 - 88 in the linked document)

V15.1 Secure Coding and Architecture Documentation

ID	Description	Level	Reference
15.1.2	Verify that an inventory catalog, such as software bill of materials (SBOM), is maintained of all third-party libraries in use, including verifying that components come from pre-defined, trusted, and continually maintained repositories.	2	Link
15.1.3	Verify that the application documentation identifies functionality which is time-consuming or resource demanding. This must include how to prevent a loss of availability due to overusing this functionality and how to avoid a situation where building a response takes longer than the consumer's timeout. Potential defenses may include asynchronous processing, using queues, and limiting parallel processes per user and per application.	2	

V15.2 Security Architecture and Dependencies

ID	Description	Level	Reference
15.2.2	Verify that the application has implemented defenses against loss of availability due to functionality which is time-consuming or resource-demanding, based on the documented security decisions and strategies for this.	2	Link
15.2.3	Verify that the production environment only includes functionality that is required for the application to function, and does not expose extraneous functionality such as test code, sample snippets, and development functionality.	2	

V15.3 Defensive Coding

ID	Description	Level	Reference
15.3.2	Verify that where the application backend makes calls to external URLs, it is configured to not follow redirects unless it is intended functionality.	2	Link
15.3.3	Verify that the application has countermeasures to protect against mass assignment attacks by limiting allowed fields per controller and action, e.g., it is not	2	

	possible to insert or update a field value when it was not intended to be part of that action.		
15.3.4	Verify that all proxying and middleware components transfer the user's original IP address correctly using trusted data fields that cannot be manipulated by the end user, and the application and web server use this correct value for logging and security decisions such as rate limiting, taking into account that even the original IP address may not be reliable due to dynamic IPs, VPNs, or corporate firewalls.	2	
15.3.5	Verify that the application explicitly ensures that variables are of the correct type and performs strict equality and comparator operations. This is to avoid type juggling or type confusion vulnerabilities caused by the application code making an assumption about a variable type.	2	
15.3.6	Verify that JavaScript code is written in a way that prevents prototype pollution, for example, by using Set() or Map() instead of object literals.	2	
15.3.7	Verify that the application has defenses against HTTP parameter pollution attacks, particularly if the application framework makes no distinction about the source of request parameters (query string, body parameters, cookies, or header fields).	2	

V16 Security Logging and Error Handling

(Refer to pages 88 - 92 in the linked document)

V16.1 Security Logging Documentation

ID	Description	Level	Reference
16.1.1	Verify that an inventory exists documenting the logging performed at each layer of the application's technology stack, what events are being logged, log formats, where that logging is stored, how it is used, how access to it is controlled, and for how long logs are kept.	2	Link

V16.2 General Logging

ID	Description	Level	Reference
16.2.1	Verify that each log entry includes necessary metadata (such as when, where, who, what) that would allow for a	2	Link

	detailed investigation of the timeline when an event happens.		
16.2.2	Verify that time sources for all logging components are synchronized, and that timestamps in security event metadata use UTC or include an explicit time zone offset. UTC is recommended to ensure consistency across distributed systems and to prevent confusion during daylight saving time transitions.	2	
16.2.3	Verify that the application only stores or broadcasts logs to the files and services that are documented in the log inventory.	2	
16.2.4	Verify that logs can be read and correlated by the log processor that is in use, preferably by using a common logging format.	2	Link
16.2.5	Verify that when logging sensitive data, the application enforces logging based on the data's protection level. For example, it may not be allowed to log certain data, such as credentials or payment details. Other data, such as session tokens, may only be logged by being hashed or masked, either in full or partially.	2	Link

V16.3 Security Events

ID	Description	Level	Reference
16.3.1	Verify that all authentication operations are logged, including successful and unsuccessful attempts. Additional metadata, such as the type of authentication or factors used, should also be collected.	2	Link
16.3.2	Verify that failed authorization attempts are logged. For L3, this must include logging all authorization decisions, including logging when sensitive data is accessed (without logging the sensitive data itself).	2	Link
16.3.3	Verify that the application logs the security events that are defined in the documentation and also logs attempts to bypass the security controls, such as input validation, business logic, and anti-automation.	2	
16.3.4	Verify that the application logs unexpected errors and security control failures such as backend TLS failures.	2	

V16.4 Log Protection

ID	Description	Level	Reference
16.4.1	Verify that all logging components appropriately encode data to prevent log injection.	2	Link
16.4.2	Verify that logs are protected from unauthorized access and cannot be modified.	2	

16.4.3	Verify that logs are securely transmitted to a logically separate system for analysis, detection, alerting, and escalation. The aim is to ensure that if the application is breached, the logs are not compromised.	2	

V16.5 Error Handling

ID	Description	Level	Reference
16.5.1	Verify that a generic message is returned to the consumer when an unexpected or security-sensitive error occurs, ensuring no exposure of sensitive internal system data such as stack traces, queries, secret keys, and tokens.	2	Link
16.5.2	Verify that the application continues to operate securely when external resource access fails, for example, by using patterns such as circuit breakers or graceful degradation.	2	Link
16.5.3	Verify that the application fails gracefully and securely, including when an exception occurs, preventing fail-open conditions such as processing a transaction despite errors resulting from validation logic.	2	
16.5.4	Verify that a "last resort" error handler is defined which will catch all unhandled exceptions. This is both to avoid losing error details that must go to log files and to ensure that an error does not take down the entire application process, leading to a loss of availability.	3	

V17 WebRTC

(Refer to pages 92 - 95 in the linked document)

V17.1 TURN Server

ID	Description	Level	Reference
17.1.1	Verify that the Traversal Using Relays around NAT (TURN) service only allows access to IP addresses that are not reserved for special purposes (e.g., internal networks, broadcast, loopback). Note that this applies to both IPv4 and IPv6 addresses.	2	Link

V17.2 Media

ID	Description	Level	Reference
17.2.1	Verify that the key for the Datagram Transport Layer Security (DTLS) certificate is managed and protected based on the documented policy for management of cryptographic keys.	2	Link
17.2.2	Verify that the media server is configured to use and support approved Datagram Transport Layer Security (DTLS) cipher suites and a secure protection profile for the DTLS Extension for establishing keys for the Secure Real-time Transport Protocol (DTLS-SRTP).	2	Link
17.2.3	Verify that Secure Real-time Transport Protocol (SRTP) authentication is checked at the media server to prevent Real-time Transport Protocol (RTP) injection attacks from leading to either a Denial of Service condition or audio or video media insertion into media streams.	2	Link
17.2.4	Verify that the media server is able to continue processing incoming media traffic when encountering malformed Secure Real-time Transport Protocol (SRTP) packets.	2	

V17.3 Signalling

ID	Description	Level	Reference
17.3.1	Verify that the signalling server is able to continue processing legitimate incoming signalling messages during a flood attack. This should be achieved by implementing rate limiting at the signalling level.	2	Link
17.3.2	Verify that the signalling server is able to continue processing legitimate signalling messages when encountering malformed signalling messages that could cause a denial of service condition. This could include implementing input validation, safely handling integer overflows, preventing buffer overflows, and employing other robust error-handling techniques.	2	

Note: *All the above content is based on the **OWASP Application Security Verification Standard (ASVS)** documentation. You can refer to the official standard here:*
- *OWASP ASVS Website*
- *OWASP ASVS Document*

*The **Level 1 (L1)** and **Level 2 (L2)** security testing requirements are applicable to **both Web and Desktop applications**. However, during Desktop application testing, **additional checkpoints** may be included to address platform-specific risks and functionalities*

References

- OWASP Top 10 Project: https://owasp.org/www-project-top-ten/
- OWASP Web Security Testing Guide: https://owasp.org/www-project-web-security-testing- guide/
- OWASP Proactive Controls: https://owasp.org/www-project-proactive-controls/
- OWASP Software Assurance Maturity Model (SAMM): https://owasp.org/www-project-samm/
- OWASP Secure Headers Project: https://owasp.org/www-project-secure-headers/
- Cheat Series
- OWASP ASVS 4.0 Testing Guide https://github.com/BlazingWind/OWASP-ASVS-4.0-testing- guide